

ETL Pipeline: Architecture, Operations & Update Guide

How the pipeline works, daily CLI usage, and adding projects/tables without AI assistance

1. ETL Directory Structure & File Map

The ETL pipeline is contained entirely inside the `etl/` directory. All business logic is configuration-driven:

File / Path	Description & Role
<code>config/projects.yml</code>	The single source of configuration. Defines schemas, tables, column mappings, coordinate SRIDs, and target ID formatting.
<code>dbt/models/</code>	SQL models: <code>stg_candidates</code> (unions & standardizes sources), <code>int_candidates</code> (collapses duplicates), <code>dup_report</code> , <code>excluded_rows</code> .
<code>dbt/macros/etl_columns.sql</code>	Jinja macros for coordinate rounding, target ID formatting, and category fallback rules.
<code>dbt/tests/</code>	Integrity tests verifying zero orphaned feedback rows, UUID5 validity, and spatial indexes.
<code>runner/run.py</code>	Python orchestrator handling locks, UUID5 generation, sequential VM numbers, and single-transaction merge gates.
<code>docker-compose.yml</code>	Defines <code>etl</code> (runner) and <code>etl-approve</code> (CLI approvals) services under the <code>--profile etl</code> flag.

2. How the Pipeline Works & Key Design Principles

The pipeline keeps `public.anomalies` synchronized with raw geophysical inspection tables (e.g. Köln picks, Wilhelmshaven Magnetic & Georadar):

```
Project Schemas -> dbt: stg_candidates -> Runner: register_ids (uuid5)
                  -> dbt: int_candidates -> Runner: merge (single transaction)
                  -> public.anomalies & <schema>.anomalie_1
```

Fundamental Rules Enforced by the Architecture:

- Immutability & No Deletions:** The pipeline user (`etl_pipeline`) has NO DELETE, TRUNCATE, or general UPDATE rights on `public.anomalies`. Production data can never be deleted.
- Approval Gated Modifications:** When existing source rows change attributes (category, depth, etc.) or shift coordinates, changes are staged in `etl.change_log` or `etl.correction_candidates`. An approver must explicitly accept them.
- Sequential VM Numbers:** Existing targets retain their VM number. New targets receive the next unused number per project (from `etl.vm_registry`), avoiding duplicates.
- Safety Verification Gates:** Before committing, the runner checks 7 integrity gates (feedback preserved, no orphan records, no duplicate VM numbers, IDs match uuid5). If any check fails, the transaction rolls back completely.
- Idempotency / Fingerprinting:** A hash of source tables and config is compared against the last run. If unchanged, the run finishes in milliseconds without unnecessary database writes.

3. Daily Operational Commands (Docker Compose)

```
# 1. Run the pipeline once (skips if nothing changed):
docker compose --profile etl run --rm etl run

# 2. Force a run (bypasses change detection):
docker compose --profile etl run --rm etl run --force

# 3. Run background scheduler (loop every ETL_INTERVAL_SECONDS in .env):
docker compose --profile etl up -d etl
```

Approving Staged Changes & Corrections:

```
# Check current pipeline status, pending decisions, and unconfigured tables:
docker compose --profile etl run --rm etl-approve

# Approve a staged attribute change (e.g. depth, category):
docker compose --profile etl run --rm etl-approve approve-change <ID...>

# Approve a coordinate correction pair:
docker compose --profile etl run --rm etl-approve approve-correction <PAIR_ID>

# Apply approved decisions to public.anomalies:
docker compose --profile etl run --rm etl run
```

4. How to Add a New Project and Table (Step-by-Step)

Step 1: Configure etl/config/projects.yml

```
projects:
  - project_id: "11-25-3333"
    project_name: "Duesseldorf Hafen"
    schema: p_11_25_3333_duesseldorf
    srid: 25832
    vm_prefix: "3333"
    sources:
      - table: anomalies_raw
        instrument: georadar
        key: id
        id_decimals: 3
        columns:
          easting: x_coord
          northing: y_coord
          depth: { column: depth_m, round: 2 }
          category: { column: category }
          layer: layer
```

Step 2: Grant Permissions to the Pipeline Role

```
docker compose --profile etl run --rm --no-deps etl setup-sql > setup.sql
docker exec -i feedback_postgres_db psql -U postgres -d nolte_geoservices -v ON_ERROR_STOP=1 < setup.sql
```

Step 3: Run Ingestion and Verify

```
docker compose --profile etl run --rm etl run --force
docker compose --profile etl run --rm etl-approve
```

Important Note on Instruments:

If your new source table uses an instrument other than georadar or magnetic (e.g. tdem), add that value to etl/dbt/models/schema.yml under accepted_values so that dbt data tests pass.